

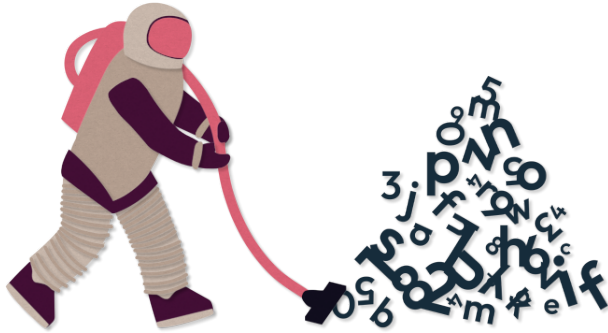
PYTHON BEST PRACTICES



**Jumping
Rivers**

CREATING CLARITY WITH YOUR DATA.

Saving organisations like yours thousands of work hours.



Jumping Rivers is an analytics company whose passion is data and machine learning. We help our clients move from data storage to data insights. Our trainers and consultants come with over 100 years combined experience in R, Python, Stan, Scala and other programming languages.

Consultancy

- Posit (formerly RStudio) certified - the only UK partner.
- We have plans for managing all Posit products. From on-demand support to full care packages.
- As a Posit reseller, we can provide free consultancy.
- Experts in both data science and data engineering.
- Creation of bespoke dashboards and Shiny apps.
- Optimisation of your algorithms.



Training

- R/Python/Stan/Scala
- The Tidyverse
- Machine Learning and Tensorflow
- Dashboards with Shiny
- Statistical Modelling
- And many more!



hello@jumpingrivers.com

@jumping_uk

"GOD IS IN THE DETAILS."

LUDWIG MIES VAN DER ROHE.

"BEWARE OF BUGS IN THE ABOVE CODE; I HAVE ONLY PROVED IT CORRECT, NOT TRIED IT."

DONALD E. KNUTH.

Introduction

When writing Python code there are often many different ways to accomplish a specific task. Having this flexibility can be advantageous, however having too many decisions to make can slow down development. When working collaboratively, the freedom in design and style choice can result in visually inconsistent code which can be difficult to parse, similarly to if each paragraph in an article had a different writing style.

When developing code we ideally want it to be clear to read, simple to maintain and easy to reproduce. Fortunately, we do not have to work from scratch. Most programming languages have developed coding standards. These give guidelines and best-practices for producing consistent and readable code.

In this course we look at some best practices for programming in Python, and how we can make these second nature by incorporating them into our every day code development.

Meaningful Names

1.1 What's in a name?

Names influence many aspects of programming. We name our variables, functions, scripts and packages. A good name can make the intention of your code clear and easy to read. A bad name can lead to confusion, misunderstanding and bugs. You might:

- forget which of `data1`, `data2`, ... contains the customer data you need;
- forget that `my_plot()` can make the box plot that you are currently writing code for;
- redefine `x` in multiple places.

Thinking of a good name for your variable can be really difficult, so the first thing to consider is if you *need* to think of a good name at all! For example, often whilst working with pandas, we might need to manipulate our data to get it into the correct format before plotting. This may lead to the creation of multiple intermediate datasets. If you don't need to save these modified datasets, it's possible to use method chaining to pass data directly to your plotting methods.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

planets = sns.load_dataset("planets")
```

Compare this code:

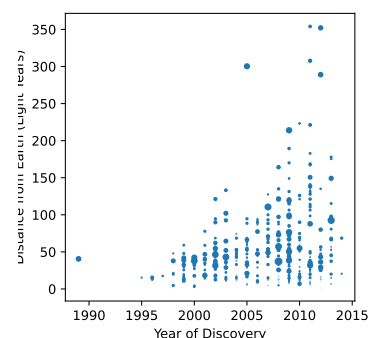
```
(
    planets
    .dropna(subset=["mass", "distance"])
    .plot(
        x="year", y="distance", s="mass", kind="scatter"
    )
)
plt.xlabel("Year of Discovery")
plt.ylabel("Distance from Earth (Light Years)")
```

With the following:

```
known_mass = planets["mass"].notna()
known_distance = planets["distance"].notna()
```



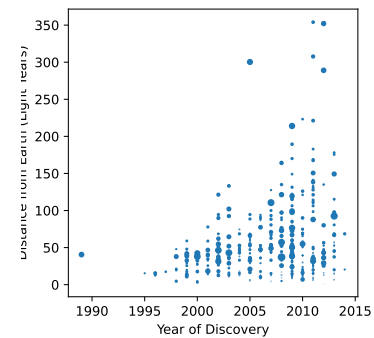
Figure 1.1: <https://xkcd.com/302/>



```
planets_filtered = planets.loc[known_mass & known_distance]

planets_filtered.plot(
    x="year", y="distance", s="mass", kind="scatter"
)
plt.xlabel("Year of Discovery")
plt.ylabel("Distance from Earth (Light Years)")
```

By using method chaining, we can avoid pausing to name the intermediary variables `known_mass`, `known_distance` and `planets_filtered`.



1.2 Naming conventions

When trying to decide a meaningful name for a variable, it is sometimes hard to think of a single-word description for its contents. For example, you've summarised the quarterly sales for a firm, should you store these as:

- `quarterlies` (will you make other quarterly summaries?); or
- `sales` (wouldn't that be more appropriate for the raw data?);

You could use several words to better contextualise your variable names. However, often in programming white space is not allowed in variable names. We could `putallthewordstogether` without spaces, but that wouldn't be very readable and might be confusing.¹

Instead, a number of naming conventions exist. The common ones are listed in Table 1.1. Within a particular language, the choice of naming convention depends on the type of object that is being named (is it a class, variable, function, type etc) and on context (is it a global variable, is it a private method). Python tends to use `snake_case` for variables, functions and methods, `PascalCase` for classes and `UPPER_SNAKE_CASE` for global variables.

Despite this convention, Python is very liberal regarding the naming syntax that is allowed. For example, you can use letters, numbers and underscore characters (`_`) in object names. The most important thing is to decide on naming conventions for your project or your team and be *consistent* throughout your code. We'll see how to enforce these naming conventions in Chapter 3.

¹ Would `cutest()` be for estimating cuts, the result of running a CU test, or just really cute?

Table 1.1: Common naming conventions in programming.

Formatting	Name
<code>two_words</code>	<code>snake_case</code>
<code>two.words</code>	<code>period.separated</code>
<code>twoWords</code>	<code>camelCase</code>
<code>TwoWords</code>	<code>PascalCase</code> , <code>UpperCamelCase</code>
<code>two-words</code>	<code>kebab-case</code>
<code>TWO_WORDS</code>	<code>UPPER_SNAKE_CASE</code>

1.3 Renaming

Consistent style is not only about naming new objects. You may also want to rename existing variables and the labels in DataFrames. A handy tool for relabelling pandas DataFrames is `pyjanitor`.²

Suppose we have a pandas DataFrame with inconsistent column names. An example is given below:

```
df = pd.DataFrame(
    {
        "timepoint": [0, 1, 2, 3, 4],
        "ROI": [0., 0., 0., 0., 0.],
```

² Note that although you install the Python library 'pyjanitor' the package you import is called 'janitor'.

```

        "subject": [0, 1, 2, 3, 4],
        "BOLD signal": [0.5, -0.4, 0.2, 0.8, -0.9],
    }
)
df.head()
#>   timepoint ROI subject BOLD signal
#> 0         0  0.0      0      0.5
#> 1         1  0.0      1     -0.4
#> 2         2  0.0      2      0.2
#> 3         3  0.0      3      0.8
#> 4         4  0.0      4     -0.9

```

Here, the columns are a mixture of upper- and lower-case and there is a column-name containing whitespace. One of the many useful methods that `pyjanitor` provides is `clean_names()`. This acts upon pandas DataFrames and returns a DataFrame with more consistent column names.

```

import janitor
clean_df = df.clean_names()
clean_df.head()
#>   timepoint roi subject bold_signal
#> 0         0  0.0      0      0.5
#> 1         1  0.0      1     -0.4
#> 2         2  0.0      2      0.2
#> 3         3  0.0      3      0.8
#> 4         4  0.0      4     -0.9

```

If a variable or function name ever seems confusing, and you can think up a better name or a better syntax, feel free to change the code. But make sure you update all the sites where that object is used.³ Don't be afraid to rename your colleagues variables either! However, if you spend time choosing a good name in the first place, you shouldn't need to spend time later renaming inappropriate names.

³ Even the most experienced developers do this all the time—during the big migration from Python 2 to Python 3 the dictionary syntax changed `dict.has_key(k)` was replaced with `k in dict`.

1.4 Write for humans, not computers

Computers can handle reading all sorts of awful code. Humans tend to be less tolerant. Clean code is code that a human can read easily. Using consistent naming conventions will help, but also try reading it out loud.

Ideally variable names and function names should be pronounceable, easy to spell, informative and easy to recall. Imagine you are talking through your code with your boss, or doing some pair programming⁴ with a colleague. How would you read the following code?⁵

```

import numpy as nums

gs5734_2021_res_mock_df = pd.DataFrame(
    {"ptCls": nums.random.choice(["ctrl", "trt"], 7,
        True),
     "yrs": nums.random.poisson(50, 7),
     "pFT": nums.random.randint(1, 15, 7)}
)

```

⁴ See Chapter 5.

⁵ GS5734 is actually the name of a type of anti-viral medication.

)

Acronyms and abbreviations are sometimes tempting, as it allows for shorter names which are easier to type. However, these contractions can be difficult to say out loud and may not make sense to someone who hasn't used them before—especially if they speak a different language.

For example, which of these two scripts is easier to say, and easier to understand?

```
def nei(s, ss, off):
    suff = s[-off:]
    ols = [x for x in ss if suff == x[:off] and s != x]
    return ols

strs = ["abc", "bcd", "cde", "fgh", "efg"]
adj = {s: nei(s, strs, 1) for s in strs}
adj
#> {'abc': ['cde'], 'bcd': [], 'cde': ['efg'], 'fgh': [],
#> 'efg': []}
```

```
def has_overlap(x, y, offset):
    x_suffix = x[-offset:]
    y_prefix = y[:offset]
    return x_suffix == y_prefix

def find_neighbours(candidate, all_strings, offset):
    prefix_overlaps = [
        string
        for string in all_strings
        if has_overlap(candidate, string, offset)
    ]
    return prefix_overlaps

string_fragments = ["abc", "bcd", "cde", "fgh", "efg"]
adjacency_list = {
    s: find_neighbours(s, string_fragments, offset=1)
    for s in string_fragments
}

adjacency_list
#> {'abc': ['cde'], 'bcd': [], 'cde': ['efg'], 'fgh': [],
#> 'efg': []}
```

Nonetheless, when a short expression name can be found, it can be very handy: a great example being the `zip()` function that converts a pair of lists into a list of pairs, reminiscent of how the two sides of a zip come together.

All in all, try saying before naming.

1.5 Name length

Good names should ideally be short, as it keeps the code easier to read. However, in the quest to make a short name, you might end up losing understandability.

Which is easier to understand below?

```
pick_random_file(path)
rndF(path)
f_rand(path)
sample_file(path)

get_todays_date()
today()

strip(strings)
trimws(strings)
str_trim(strings)
chomp(strings)
```

Another thing to consider in terms of name length is how *searchable* your name is. Let's say we need to find all instances of our variable which defines the date of an event. Which is easier to search for `d` or `start_date`?

If there is a trade off between a short variable or an understandable variable name, we would always opt for the longer, more interpretable variable name. It's better to be clear than short. And remember in most good IDEs⁶ you can use tab to auto-complete long names. A good rule of thumb is that the length of a name should correspond to the size of its scope.

⁶ Integrated Development Environment, a programme you use to write scripts, e.g. PyCharm or Visual Studio.

1.6 Function names

When naming functions, there are some additional things to consider beyond the general advice above. Functions perform an action (e.g., computing, retrieving or saving something), and as such, their names should typically start with a verb: `create_graph()`, `update_db()`, `clean_data()`. But, any given action may have multiple synonyms. So, which of the following would be the most appropriate name for your data import function?

```
load_clients()
import_clients()
fetch_clients()
retrieve_clients()
```

It doesn't really matter. What does matter is *consistency*:

- if you use `load_clients()` to import the data for your clients,
- ... then use `load_products()`, `load_suppliers()` (and so on) for importing other datasets,
- ... and use `filter_clients()`, `save_clients()` (and so on) for other functions pertaining to client-related datasets.

Having consistently-named functions makes it easier to find the right function when you need to use it.

Nonetheless, accurate naming is more important than consistent naming: if your function is called `check_url_format()` it should *only* check that the URL(s) that you pass to it have a valid format—it shouldn't check if a web page exists and it shouldn't check whether a set of email addresses are correctly formatted.

Cute names and in-jokes might seem a fun idea at the time, but when you look at your code later, you might struggle to work out the intention of the function. An example of this might be the numpy function `ptp()`. Though once you know that this runs a “peak to peak” algorithm (though that in itself is ambiguous; it finds the differences between the min and max values in a numpy array) the name may be more clear.

The `verb()` and `verb_noun()` function-naming convention is just a recommendation. Python contains many examples of well-named functions that do not conform to this style which you could consider replicating in your own code. For example:

- `barenoun()`
 - A really non-specific verb may add little value to a function name, especially where a well-established value or property is returned
 - Compare `compute_mean()` / `get_length()` to `mean()` / `len()`.
- `prefix_verb()` and `prefix_verb_noun()`
 - When the verb you want to use is already in use,
 - or, you have a suite of functions that all apply to the same data type
 - if you do end up with functions named like this, consider grouping them together instead ...
- `module_or_class.verb()`
 - In Python you could define a module, package or a class and use this as a prefix
 - So, although there is a base `format()` function you could still have a `format()` function that is specific to your module or class.
- if you have defined a custom class, you could redefine a Python operator
 - for example, could you define an addition operation for your class?

1.7 File names

Files are another important object that we deal with when programming. When creating or renaming files, these ideas may help ensure the future usability of those files:

- It can be helpful to store metadata as part of the filename e.g. `2016-05-alaska.csv`
- If you use manuscript location in figure filenames e.g. `fig_3_a.png`—be aware things might change!
- Use filenames that are portable across operating systems:
 - Never use spaces in filenames, this aids portability to Linux and OS-X
 - Use lower-case names (since Windows directories are case-insensitive by default)
- Use consistent file name extensions (e.g., use `*.jpeg` rather than `*.jpg` or `*.Jpeg`)

1.8 Clutter

To recap: when it comes to naming objects, there are three main forces in play. A name should:

- describe the purpose of the object;
- be as short as possible;
- be consistent with the other object-names in a project.

... but the first two qualities compete with each other.

Sometimes the path to creating concise, meaningful names involves removing clutter.

Unnecessary prefixes / suffixes

Suppose `outages_table_2021` is the name of a pandas `DataFrame` containing information about power outages in 2021 in your local area.

The name is self-explanatory, but is rather long. Consider each part of the name:

- `outages` : the table is about power-outages, so this part seems necessary
- `table` : we know (or can determine) that this is a `DataFrame`, so including `table` in the name is superfluous.
- `2021` : whether this is of value depends on context—is there an `outages_table_2020` in the same script? If not, maybe we can remove `2021` from the variable name. If so, we ought to keep the year in the name for now.

So, if part of a variable name is uninformative or is obvious from the context, that can usually be removed.

Clutter isn't always clutter

Don't always view prefixes or long object names as clutter—sometimes they indicate a missing abstraction.

Suppose multiple variables with a shared prefix were mentioned in a script:

- `dataset_raw`
- `dataset_clean`
- `dataset_train`
- `dataset_test` .

You could view the `dataset_` prefix as clutter and rename these variables (`raw`, `clean`, `train`, `test`). This isn't always wise. Shared prefixes (or suffixes) sometimes indicate when a set of variables have a shared context. Removing that prefix may take away an important mental cue and slow down your future coding. Here, rearranging those datasets into a larger dataset (e.g., by putting them in a class or a dictionary) that makes the shared context more explicit may have more value (though refactoring in this way⁷ is out of scope for this chapter).

Tangentially, if you have defined functions that have overly-long or highly-specific names, this might also indicate a missing abstraction. Rather than defining `read_outages_2020_data()` and `read_outages_2019_data()`

⁷ Refactoring is the process of rewriting code without altering its behaviour.

you should aim to define `read_outages_data(year)` . Identifying when to generalise a function is rarely so simple, but a rule of thumb is “Three strikes and refactor”.⁸ That is, if you find yourself writing similar code multiple times, try to write a function that generalises that code. Again, this is beyond the scope of a chapter on renaming. But, you never know, the generalised function might even be easier to name.

⁸ Fowler, Refactoring.

Modular Code

Modularity, in addition to picking good names, is a key ingredient when trying to improve the readability of code. Modularity is about two main ideas:

1. splitting your programming challenges into smaller components or tasks, and
2. collecting together those components that have related purposes.

By separating a program up in this way you help yourself because a small, independent, task:

- is easier to describe (finding pre-existing solutions is simplified);
- has fewer moving parts and less to think about (solving it yourself is simplified);
- has solutions that are easier to verify (checking it with colleagues is simplified);
- may arise in several projects (so the solutions may be reusable);
- can be worked on by other members of a team (so a solution may arise faster).

Ultimately, when scanning through a script or report that has been nicely subdivided, the aim of the code is more apparent and easier to describe to others. So, before you start programming

- *Think*: What are the main datasets, tasks and outputs involved here? For those tasks, which pose the greatest challenges (and can they be split into smaller challenges)?
- *Draw*: Make a flowchart of a collection of the tasks—connect in any datasets that are required or produced
- *Discuss*: Has a colleague (or yourself), or a pre-existing Python package solved the challenges involved here?

In Python, the main approaches for separating-up parts of a program or project are through writing functions, classes, modules and packages.

The most fundamental problem in computer science is problem decomposition. John Ousterhout, 2018, “A Philosophy of Software Design”.

2.1 Write for humans, not computers

There are many ways to generate a particular value using code. Each of these expressions generate a list of the integers between 1 and 10 (inclusive).

```
list(range(1, 11))  
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
[i+1 for i in range(10)]
```

When you can see several different ways to write the same code, how should you proceed? You could choose the version of the code that:

- runs the fastest
- uses the least computational memory
- is the easiest to modify
- requires the least typing
- is the easiest to understand.

When faced with this choice, you should always aim to write code that is as easily understood as possible. So, although optimising for readability might mean that your code sacrifices speed or memory, the benefits gained when you come back to read your code are vast.¹

Also, humans have a much smaller working memory than computers. Trying to hold lots of new information in our heads can be challenging. By chunking functions into smaller, digestible pieces, it's generally easier for others to follow the workflow.

"We are constantly reading old code as part of the effort to write new code." Robert C. Martin, 2009, "Clean Code".

¹ If you ever find it hard to follow some code, when re-reading it, then it might be worth investing some time to rewrite it. But be careful to ensure that your new program works the same afterwards.

2.2 Don't Repeat Yourself (DRY)

We often repeat ourselves in code. Sometimes this happens by accident: you might not realise some previously-written code exists that is very similar to your newly written code. Indeed, since two blocks of code can look different but compute the same expression, you might not even realise that your new code duplicates some old functionality. Then, sometimes it happens on purpose. A couple of lines here and there doesn't really matter, but when you find yourself copy and pasting substantial code blocks within a report, or between scripts or projects then something is amiss.

The problems with duplicated code are that it makes code harder to read, more time consuming to maintain, and increases the chances of bugs. For example, let's say you're studying different species of penguins and want to compare the widths and lengths of their bills. You might want to create a scatter plot for each different penguin species. First we will load data on two different penguin species.²

```
import jupyterbestprac as jrbp

penguins_chinstrap, penguins_adelie = (
    jrbp.data.load_penguins()
)
```

The following code converts the measurements from millimetres to centimetres, and then creates a scatter plot.

```
import matplotlib.pyplot as plt

penguins_chinstrap[["bill_length_cm", "bill_depth_cm"]] = (
    penguins_chinstrap[["bill_length_mm", "bill_depth_mm"]]
    / 10
)
```

² The data used in this example comes from the Antarctica Long Term Ecological Research Program and was obtained from the {palmerpenguins} R package. For simplicity, we've tweaked some of the data and added it to the course package.

```
plt.scatter(
    penguins_chinstrap["bill_length_cm"],
    penguins_chinstrap["bill_depth_cm"],
)
plt.title("Chinstrap Penguins Bill Length and Depth")
plt.xlabel("Bill Length")
plt.ylabel("Bill Depth")
```

If we wanted to create the same plot for the Adelie penguins, we could copy and paste all the code—changing the word Chinstrap to Adelie.

This is dangerous for a couple of reasons. We might remember to change the data set from `penguins_chinstrap` to `penguins_adelie` but forget to change the penguin species in the title (or vice versa). We might have a typo in our original code (such as typing `plt.xlab()` instead of `plt.xlabel()`) which gets copied across every time we paste our code. Finally, if we wanted to change the plot style, perhaps changing the colour of the points from blue to green, we would need to change this in multiple places (and not make any typos!). Another issue with duplicating this code, is it will become much harder to read—and therefore harder to spot any mistakes.

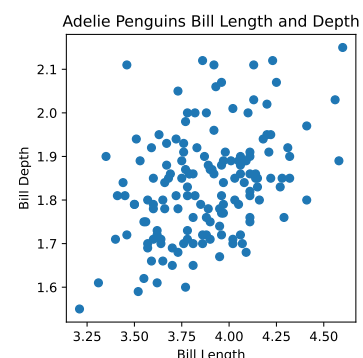
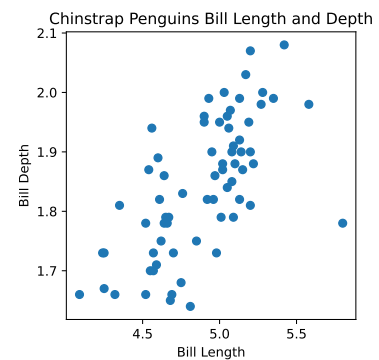
Any time you find yourself using copy and paste, take a pause and consider if there is a way for you to not duplicate your code. The best course of action depends on how the code is being reused. When you find this kind of duplication, ask yourself which commands are common to the different code-blocks and which vary the most. In Python, the simplest routes for de-duplicating code are to introduce variables and to define functions.

If in the different code-blocks, you are using the same processing steps on several different datasets you may benefit from introducing a function to handle the processing pipeline. In the penguin example, we convert the measurements from cm to mm for all three datasets. We could move these two lines into a single function `convert_to_cm()`.

```
def convert_to_cm(data):
    data[["bill_length_cm", "bill_depth_cm"]] = (
        data[["bill_length_mm", "bill_depth_mm"]] / 10
    )
    return data

penguins_adelie = convert_to_cm(penguins_adelie)
plt.scatter(
    penguins_adelie["bill_length_cm"],
    penguins_adelie["bill_depth_cm"],
)
plt.title("Adelie Penguins Bill Length and Depth")
plt.xlabel("Bill Length")
plt.ylabel("Bill Depth")
```

If there are minor variations in your code you can use variables to handle the variety. If we wanted to move the entire plotting code into a function,



we would need to account for the fact that each plot requires a unique title. We can do this with a function argument.

```
def plot_bill_scatter(data, species):
    plt.scatter(
        data["bill_length_cm"], data["bill_depth_cm"]
    )
    plt.title(f"{species} Penguins Bill Length and Depth")
    plt.xlabel("Bill Length")
    plt.ylabel("Bill Depth")

penguins_adelie = convert_to_cm(penguins_adelie)
plot_bill_scatter(penguins_adelie, "Adelie")
```

2.3 Writing good functions

There are a number of guidelines that will help when designing and using functions or methods.

Structure

A well written function should “Do one thing, do it well, and do it only”.³ If you have written a function `regress()` that performs linear regression, that function shouldn’t also generate plots of the dataset. When naming your function, if you cannot create a succinct name it may simply be that the function is doing too many things and should be split up.

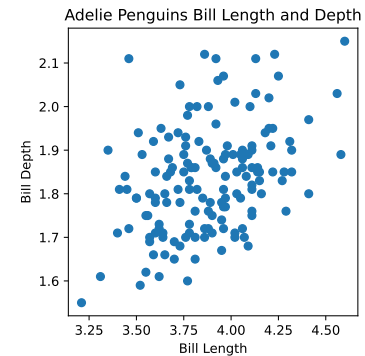
Ideally, function definitions should be small. To some, *small* might mean that the main body of the function can be read without scrolling on a laptop screen, to others it might mean that the function body contains only a handful of variable declarations and function calls. This may be difficult to achieve, but it is only a rule-of-thumb.

More important is that a function should be readable. While reading a function, you may find there is a lot of low-level complexity in there that makes it hard to follow why the function is structured as it is (including things like validity checking). If so, consider moving that low-level work into a new function and calling it.

For example, the following function has boilerplate check code:

```
def convert_age(age_days, units="years"):
    assert isinstance(age_days, (int, float)), \
        "Ages should be numeric"
    assert age_days >= 0, "Ages should be non-negative"
    assert age_days <= 150 * 365, "A bit too old?"
    assert units in ["hours", "days", "years"]

    if units == "years":
        age = age_days / 365
    elif units == "days":
        age = age_days
    elif units == "hours":
```



³ Error handling is one thing!

A reasonable upper limit, is 100 lines of code in a function.

Chunk 2.1: Python’s `assert` statement can be used to help detect problems early-on in a programme’s execution. `assert condition` triggers an error if `condition` evaluates to `False`. Additionally, a message can be printed if the `condition` evaluates to `False`, by using the pattern: `assert condition, "print if False"`.


```

        age = age_days * 24

    return age

```

We could move the first few lines to

```

def check_valid_age(age):
    # Assumes the age is in days
    assert isinstance(age, (int, float)), \
        "Ages should be numeric"
    assert age >= 0, "Ages should be non-negative"
    assert age <= 150 * 365, "A bit too old?"

def convert_age(age_days, units="years"):
    check_valid_age(age_days)
    assert units in ["hours", "days", "years"]

    if units == "years":
        age = age_days / 365
    elif units == "days":
        age = age_days
    elif units == "hours":
        age = age_days * 24

    return age

```

This would significantly simplify the function.

However in Python, functions are more usually written in a more lenient way. A digestible summary of how you write Python functions is “it is easier to ask for forgiveness than permission” (EAFP). In this style, your function should attempt to perform its action without doing all that prevalidation. If the function fails, you can subsequently handle anything that went wrong.

So, a more ‘pythonic’ way to write the above function might be as follows:

```

def convert_age(age_days, units="years"):
    age_multipliers = {
        "years": 1 / 365,
        "days": 1.0,
        "hours": 24.0
    }
    try:
        return age_days * age_multipliers[units]
    except KeyError:
        raise ValueError(
            "'units' should be 'years', 'days' or 'hours'"
        )
    except TypeError:
        raise TypeError("'age_days' should be numeric")

```

A virtue of this style is that the ‘happy path’ of the function is condensed

and clearly visible at the top of the definition. But, we no longer disallow negative or extremely large ages—perhaps it wasn't the responsibility of an age-conversion function to reject these in the first place.

Contract

The *contract* defines what a function or method does (not how it does it) and the interface of that function (the data-types that it receives and returns, the parameter names). Fight the impulse to write the body of your function—think about the contract of that function first.

- If the function acts on numeric values, should it accept negative values?
- If it accepts `DataFrame`s as input, does it need to know which columns of that `DataFrame` it should act upon?
- If you have written other functions with similar functionality, are the arguments named consistently, do they return consistent data-types?
- Should the output of the function be of the same structure as the input?
- If the user passes in invalid input, should the function throw an error?

For each argument that the user can pass into your function, there will be extra code to write and the resulting function will be more complicated and harder to test. So it simplifies your life if your functions have as few parameters as possible. But, if adding extra parameters increases the applicability of your function, reduces the need for extra functions, or reduces the burden on the user then the change can be justified.

Take a second to think about 'pandas'. One of the main reasons the library is so successful is that the key methods have the same output as their input. This makes it very easy to use.

The contract is very important. The minute someone else starts using your code, if you change something you have now given them more work.

No side-effects!

A pure function is one that has no side-effects and is self-contained. These are really useful properties that make it easier to reason about your code.

Side-effects include things like writing to files, printing to the command line, setting global options or modifying values outside of the scope of the function. A function that is self-contained is one where, for any given input arguments, it always provides the same output. So a function that uses a random number generator, that depends on a global configuration option or other value that isn't passed into the function as an argument, would not be pure.

Because they neither influence, nor are influenced by global state, pure functions play nicely with the rest of your code.

You will, however, need to access files, databases, webpages and random numbers when programming with data, so it is hard to eliminate impure function calls from your code. But, when writing functions that do these things, if you are able to separate the impure behaviour away from calls to a pure function you'll benefit.

For example, suppose you have a function that reads some data and processes it. By separating the processing steps into a pure function, you benefit—you are no longer tied to the data-storage format and if you've already imported the data for some other reason you can pass that into the pure function. For example, if we move line 2 of this function

```
def f(path):
    raw_values = read_the_data(path)
    processed_values = do(some(thing(to(values))))
    return processed_values
```

into a separate function

```
def process_the_data(x):
    processed_values = do(some(thing(to(x))))
    return processed_values
```

This makes it easier to move from a CSV file to a database to Dropbox.

Write generalisable code

If you find yourself writing a function that could *only* be used for a particular project, ask yourself if there is a way to make that function *slightly more* general. For example, if you were to write a function to load and clean-up data from the file `./data/health.csv`, which of the following would be better:

```
import os.path

def import_the_data():
    path = os.path.join("data", "health.csv")
    raw_data = read_the_data(path=path)
    return clean_the_data(raw_data)

def import_the_data(path=os.path.join("data", "health.csv")):
    raw_data = read_the_data(path=path)
    return clean_the_data(raw_data)
```

We use `os.path.join()` to ensure file-path portability.

tent code style,⁴ you will find it easier to focus on what the code *does* rather than how it *looks* when you come back to read it again.

⁴ Shockingly, coders can be pedantic at times. So having a single uniform style avoids the “styling wars”.

3.2 *Automation to the rescue*

Depending on the team and project, sticking to a style guide could feel like a burden: writing code may take you longer (you are both solving a problem, and thinking how to style your solution), and reviewing code may both take longer and identify seemingly minor issues.

Automation can prevent code-style from becoming a burden. This has a two-fold benefit: people find it less intrusive when an automated tool tells them they’ve misaligned their code than when their colleagues do, and, code-reviews become more pleasant since you and your colleagues can focus on the high-level design of your code.

There are two main types of code-style automation tools:

- Code formatters (stylers)
 - These tools *do* change your source code;
 - They fix things that won’t influence the running of your code, e.g., white-space, line-wrapping, quoting;
 - Limited configuration options;
 - Examples: R {styler} and Python {black}.
- Code analysers (linters)⁵
 - These tools check your code against a style-configuration for your project;
 - They do not modify your code;
 - They can identify the same issues that are fixed by formatters;
 - But also identify things that require human-input to fix them: variable-names, use of undesirable functions, absolute-paths, system calls etc.;
 - Highly configurable;
 - Examples: R {lintr}⁶ and Python {pylint} & {flake8}.

⁵ Lint being the loose fluff that comes away from your jumpers...

Many IDEs and text editors (RStudio, VS Code, PyCharm) provide integrations for these tools. But, when and where these tools are used is up to you—to ensure they are used, it is better to set up an automated pipeline. A typical work flow is to run code-formatters on-the-fly (e.g., whenever a file is saved) and to use code-analysers as part of a continuous integration pipeline, so that running this step doesn’t depend upon the local computer set up of each member of your team.

⁶ All Jumping Rivers course notes are put through a lint check, to ensure a consistent style across courses. Our trainers usually curse the linter.

Be consistent

To maintain the aesthetic consistency of the code in your project, you will need to decide on a code style, document it, and then encode it in such a way that it can be enforced upon the project (by configuring a formatter and analyser).

However, you shouldn’t reinvent the wheel. Instead, adopt an existing style guide and tweak as necessary. Although Python allows you great freedom in how you name your variables, classes and functions, the Python community has adopted a set of conventions for the low-level styling of code: PEP8. PEP8 includes many recommendations, such as:

- Use 4-spaces per indentation level;
- `import` s should be placed one-per-line at the top of your file;
- functions and methods should use `snake_case` , classes `UpperCamelCase` .

It is a good idea to use a well-defined style guide, like PEP8, in your own projects for several reasons: the style guide is well-documented; many open-source projects already use that style-guide, and configuration options for the styling tools can readily be found by looking within those projects. Indeed, the default options for both `{black}` and `{flake8}` are based on PEP8.

Running your own linter

Even with a styling baseline set by PEP8, there still remains a lot of flexibility in how your team could layout their code. By being able to run and configure linters and stylers you can ensure that your style guide is adhered to throughout the lifetime of your project.

Chunk 3.1 shows a small Python script with formatting inconsistent with PEP8 (amongst other things, there is a 2-space indent in the function body and the function name uses `camelCase`). The tool `flake8` can be used to compare this script to most of the PEP8 conventions (though you may need to install some extensions).

```
# script-dirty.py
a= 123
b = 456
from numpy import *
def myFunc(x):
    '''
    some function
    '''
    return mean(x)/sd(x)
myFunc([a,b])
```

Chunk 3.1: The content in the file `script-dirty.py` does not adhere to the specifications outlined in PEP8.

If we run `flake8` on this script from the command line we get the following output:

```
flake8 script-dirty.py
#> script-dirty.py:2:2: E225 missing whitespace around
#> operator
#> script-dirty.py:2:3: E222 multiple spaces after operator
#> script-dirty.py:4:1: F403 'from numpy import *' used;
#> unable to detect undefined names
#> script-dirty.py:4:1: E402 module level import not at top
#> of file
#> script-dirty.py:5:1: E302 expected 2 blank lines, found
#> 0
#> script-dirty.py:5:6: N802 function name 'myFunc' should
#> be lowercase
#> script-dirty.py:6:3: E111 indentation is not a multiple
#> of 4
#> script-dirty.py:9:3: E111 indentation is not a multiple
```

```
#> of 4
#> script-dirty.py:9:10: F405 'mean' may be undefined, or
#> defined from star imports: numpy
#> script-dirty.py:9:18: F405 'sd' may be undefined, or
#> defined from star imports: numpy
#> script-dirty.py:10:1: E305 expected 2 blank lines after
#> class or function definition, found 0
#> script-dirty.py:10:10: E231 missing whitespace after ','
```

Each line of the output indicates a diversion between the format of the script, and the format expected by the flake8 defaults—for example, the assignment on the second line should have a single space either side of the equals sign. Many of the low-level concerns can be fixed using a code-formatter. Here we will use black (again, from the command line):

```
black script-dirty.py
```

black will modify the script in place. After modification the script will look like:

```
# script-dirty.py
a = 123
b = 456
from numpy import *

def myFunc(x):
    """
    some function
    """
    return mean(x) / sd(x)

myFunc([a, b])
```

Although this has made the script look more uniform, it still fails the flake8 linter. However, the remaining issues flagged will all require human input to resolve:

```
flake8 script-dirty.py
#> script-dirty.py:4:1: F403 'from numpy import *' used;
#> unable to detect undefined names
#> script-dirty.py:4:1: E402 module level import not at top
#> of file
#> script-dirty.py:7:6: N802 function name 'myFunc' should
#> be lowercase
#> script-dirty.py:11:12: F405 'mean' may be undefined, or
#> defined from star imports: numpy
#> script-dirty.py:11:22: F405 'sd' may be undefined, or
#> defined from star imports: numpy
```

To fix these final lints, we might use `import numpy as np` at the top of the script, explicitly namespace the `numpy` functions that are used (e.g.,

`np.mean(x)`) and rename our custom function `my_func()` (or something descriptive).

3.3 Layout of scripts

There should be a logical order to the structure of your scripts. You should decide on conventions for ordering these, however there are some general tips about packages, functions and spacing.

Handling dependencies

In R and Python, scripts are ran from top-to-bottom. When a code block runs, all the dependencies of that code block (the variables, functions, classes and packages that it requires) must already be available. This constrains the structure of your files. Since each package must be loaded before it is used, it makes sense to put *all* package-loading code at the very start of a script. Organising package-loading code like this is standard practice in these languages: it gives the reader a quick overview of the external dependencies of the script, and since these lines run first, means that a script will fail early if any package-dependencies are missing.

A common anti-pattern in Python is to load all of the contents of a library using `from <libraryName> import *`. This style of import allows you to use functions and classes from a library without having to namespace the function / class.

```
# Don't do this - it attaches everything that numpy
# exports to your session
from numpy import *
mean([1, 2, 3, 4])
```

The problem with this style of import, is it may attach hundreds of object names to your session, making it very easy for name clashes to arise. For example, the `var` function from `numpy` will overwrite the base python `var` function if `numpy` is imported. A better solution is to import the library and then access the functions you need with explicit namespacing.

```
# Here we import the library numpy, rather than all of its
# contents, then use just those functions/classes that we
# need
import numpy as np
np.mean([1, 2, 3, 4])
```

Consider the other internal and external dependencies within a script. The other external dependencies might include user-defined options, files, databases and web pages that are accessed while running the code—by handling these in a single place, early during the running of your script, you help the reader understand the inputs, outputs and requirements of that script.⁷

⁷ The running of the script may be improved by this too: imagine running a lengthy computation before checking whether you can write the results to a given directory.

Organising functions

Similarly, each function must be defined before it is called. As such, it is common to put all function definitions together in a single place towards the start of a script. A sensible choice would be to collect together functions by related tasks, or by the type of dataset that they work upon, so that navigating your functions is simplified.

Collecting your functions together still leaves a lot of style choice. Note that functions aren't *called* if they are used in the definition of another function. Now, we know that this code is invalid (because you can't call a function before it is defined):

```
double_me(2)
```

```
def double_me(x):
    return x * 2
```

But, this code is perfectly valid, because `double_it()` is not *called* during the definition of `triple_it()` (it is only called when `triple_it()` is evaluated):

```
def triple_it(x):
    return double_it(x) + x
```

```
def double_it(x):
    return x * 2
```

```
triple_it(3)
#> 9
```

So, when organising your functions in a script or package, you might organise them in a top-down way:

```
def process_it(x):
    cleaned_up_data = clean_it(x)
    summarised_data = summarise_it(cleaned_up_data)
    return summarised_data
```

```
def clean_it(x):
    """
    How we clean the data
    """
    pass
```

```
def summarise_it(x):
    """
    How we summarise the data
    """
    pass
```

Or, you might organise them in a bottom-up way:

```
def summarise_it(x):
    """
    How we summarise the data
    """
    pass

def clean_it(x):
    """
    How we clean the data
    """
    pass

def process_it(x):
    cleaned_up_data = clean_it(x)
    summarised_data = summarise_it(cleaned_up_data)
    return summarised_data
```

While working in an analysis script or notebook, you may define lots of related functions or several class / method definitions. That script may be getting pretty long. It is a good idea to separate out parts of your code into stand-alone files, called modules, with the extension `.py`. These modules can be imported into your analysis script. By doing this, your code becomes reusable: those same functions and classes can be used across several analysis scripts⁸.

In the example above, you might put the definition for `process_it()` into the text file `data_workflow.py` (along with its helper functions). Then from your Jupyter notebook you can `import data_workflow` and use the main function like `data_workflow.process_it(x)`.⁹

Using vertical space and declaring variables

When it comes to code layout, vertical spacing can be useful to help the user follow the code structure. Similar concepts and functions can be located close together, whilst code that is unrelated can be kept further apart.

Suppose we need to define some variables in a script, for example, defining a global colour for use in plotting `jumping_rivers_blue = "#516e7a"` or specifying a common file path `path_to_data = "data/user-data"`. It's a good idea to define these variables near the code where they are used. If the variables are used throughout the script, you might choose to define them either near the first instance of their usage, or at the top of the script.

For example the following code works but could be spaced out and re-ordered to be easier on the eye.

```
import os
import jupyterbestprac as jrbp
try:
    os.mkdir("penguins")
except FileNotFoundError:
    pass
```

⁸ Indeed, any non-trivial code that is duplicated between multiple analysis scripts is a good candidate for placement in a module.

⁹ If you modify a local module during exploratory work, the changes you make won't be automatically available to a running Jupyter notebook that has already imported that module. You can use `importlib.reload(the_module)` to integrate any newly added code.

```

import pandas as pd
penguins_chinstrap, penguins_adelie = (
    jrpb.data.load_penguins()
)
colours = {
    "Adelie": "#FF8C00", # (255, 140, 0),
    "Chinstrap": "#A020F0", # (160, 32, 240),
    "Gentoo": "#008B8B" # (0, 139, 139),
}
penguins = pd.concat([penguins_chinstrap, penguins_adelie])
import matplotlib.pyplot as plt
penguins_male = (
    penguins
    .query("sex == 'male'")
    [
        "species", "bill_length_mm", "body_mass_g"
    ]
)
path = "penguins"
penguins_male.plot(
    "body_mass_g", "bill_length_mm",
    c=penguins_male["species"].map(colours),
    kind="scatter"
)
penguins_male.to_csv(os.path.join(path, "males.csv"))
plt.show()
plt.savefig(
    os.path.join(path, "males-mass-x-bill_length.png")
)

```

We can group the `import` calls together, chunk the pre-processing, the plotting and the saving. By separating these sections of code using vertical space (new lines) we can help the reader quickly search through the code and find the relevant lines.

We can also deduplicate the use of the `path` variable and define the constants that are used within the script near to the start of the script. This helps when repurposing a script for other uses, though you may prefer to define variables close to where they are used, for easier debugging.

```

import os
import pandas as pd
import matplotlib.pyplot as plt
import jrpybestprac as jrpb

path = "penguins"
files = {
    "table": os.path.join(path, "males.csv"),
    "figure": os.path.join(
        path, "males-mass-x-bill_length.png"
    ),
}

```

```

colours = {
    "Adelie": "#FF8C00", # (255, 140, 0),
    "Chinstrap": "#A020F0", # (160, 32, 240),
    "Gentoo": "#008B8B", # (0, 139, 139),
}

try:
    os.mkdir(path)
except FileExistsError:
    pass

penguins_chinstrap, penguins_adelie = (
    jrpb.data.load_penguins()
)
penguins = pd.concat([penguins_chinstrap, penguins_adelie])

penguins_male = (
    penguins
    .query("sex == 'male'") [
        ["species", "bill_length_mm", "body_mass_g"]
    ]
)

penguins_male.plot(
    "body_mass_g",
    "bill_length_mm",
    c=penguins_male["species"].map(colours),
    kind="scatter",
)
plt.show()

penguins_male.to_csv(files["table"])
plt.savefig(files["figure"])

```

Ultimately, how you organise the large-scale structure of your scripts is up to you. But since there are few automated tools available to maintain this global structure, it is important that you write in a way that means your design choices will be continued as the code develops.

3.4 Code comments

Comments and function-level documentation provide extra information about your source code. Your code would work perfectly well without any comments or documentation, however, well placed comments can help a programmer have a better understanding of your code. Comments can provide a means to elaborate:

- what the code does; and
- why the code was written the way it was.

These are details of the *contract*, *constraints* and *design* of the code: aspects of your code that change relatively slowly.

A mistake that many people make is to comment on the *mechanics* of how their code works. But that's what the code is for, so this kind of comment just duplicates information that could be read from the source files. Also, since code evolves over time, unless efforts are made to check comments whenever the code changes, these mechanistic comments can quickly become out-of-date.

Code-comments can be very useful, for example, alerting a programmer that the model fit code is slow to run. However, comments can also be over-used and they can get out-of-sync with the code they are supposed to clarify. Before writing a comment, ask yourself the following questions:

- Would this information be better in the function documentation?
- Is what I'm typing accurate?¹⁰
- Is my comment encapsulated by my version control (e.g. author, date, change log)
- Can I refactor my code so it's clearer, making the comment redundant.

¹⁰ No comments are better than an inaccurate comment!

If you have written a paragraph of comments to explain the code, you probably need to refactor the code instead. Sometimes, splitting up functions into smaller components with clear names will mean the comment is no longer required.

Commented code

Why do we have the tendency to comment out code instead of just deleting it? The reason is that we don't want to lose a piece of code that might come in handy in the future. It all boils down to fear of losing information. But commented out code will get in the way more than it will help you, and it's better off being deleted.

Commented out code is distracting when you are reading a script, making it harder for your brain to build a mental model of what the code is doing. Also, as our commented out code gets older, the higher the chances it no longer fits when uncommented which might result in failing code or bugs.

If you are using version control software such as git, you can safely delete those lines of code, knowing you can always retrieve them later if you need them. If that isn't an option, you might want to consider pulling the commented code into a knowledge repository associated with the code base.

Docstrings

In addition to variables, we have seen that a Python program can be decomposed into functions, classes and modules. Also, we noted that comments are good when they explain the purpose of a piece of code, but may have little or no value when they explain how that code works (or used to work).

In Python, there is a definitive place to add code-comments for functions, classes and modules—in docstrings. These are strings that are added as the first line of the relevant entity. Suppose the following was the contents of a Python module, containing a class and a function. The different types of

docstring are indicated.

```

"""
Python module for working with circular shapes.

This is a module docstring
"""

class Circle:
    """
    A simple class for representing circles

    A class docstring
    """

    pi = math.pi

    def __init__(self, centre, radius):
        self.centre = centre
        self.radius = radius

    def area(self):
        """
        Computes the area of the circle

        A method docstring
        """
        pass

def any_overlap(circle1, circle2):
    """
    Determines whether two circles overlap

    Uses algorithm ZZZ
    A function docstring

    Parameters
    -----
    circle1 : Circle
        A circle object
    circle2 : Circle
        A circle object

    Returns
    -----
    boolean
        True if the two circles intersect. False otherwise.
    """
    pass

```

The great thing about docstrings is, after you have defined the relevant objects in your Python session (by importing a module, or defining a function), you can access help pages on that object. For example, using `help(Circle)` . This makes your explanatory comments available both to developers who are extending your code, and to users of your code as well.

Reproducible Workflows

A minimal standard for data analyses is that they be reproducible: that the code and data are assembled in a way so that another person can recreate all of the results. Adopting a workflow that will make your results reproducible will ultimately make your life easier; if a problem arises, or your data updates, it will be much easier to correct.

Organising analyses so that they are reproducible is not easy. It requires diligence and a considerable investment of time, but *partially reproducible* is better than not reproducible at all. Just try to make your next project better organised than the last.

You can think of different levels of reproducibility.

1. You can rerun the analysis next week on the same computer
2. You can rerun the analysis on a different computer
3. Someone else can rerun the analysis (today)
4. Someone else can rerun the analysis in 5 years time

4.1 Organise your files

Perhaps the most important step to take towards reproducibility is to be organised. Each project should live in its own directory, which is named after the project. This directory should contain *everything* needed to reproduce your work, including all scripts, data, documentation and results.

A mistake that many data scientists make is to include the full path to a file or directory on their machine in their analysis code.

```
import os
import pandas as pd

project_dir = "".join(
    "C:/Users/bill/Documents/",
    "analysis/path/to/CoolProject"
)
files = {
    "cats": os.path.join(
        project_dir, "data", "cat-species.csv"
    )
}

felines = pd.read_csv(files["cats"])
```



Figure 4.1: <https://xkcd.com/1077/>

Although this might work for a while, if you move your files around, or try to share them with a colleague those line of code won't work, because the file path won't exist.

A better solution would be to use *relative file paths*. Rather than specifying the absolute path to the cat dataset, you could specify it relative to the root-directory of your analysis project and run all your analysis scripts from that directory.

```
files = {
    "cats": os.path.join("data", "cat-species.csv")
}

felines = pd.read_csv(files["cats"])
```

Now, when a colleague opens a copy of your project on their computer the relative path to that dataset will be the same, even though they may store the analysis project in a different location.

4.2 Documentation

Documentation of code occurs at several scales. Suppose you have written a new package or an analysis project. You might write:

- a README, website¹ or publication to help new users get an overview of the purpose of the project;
- tutorials and other long-form documentation, to help users understand how to use your code to achieve an overall goal;
- help pages to explain how to use specific components of the code (specific functions or classes);
- comments to help programmers understand the inner workings of your code.

¹ sphinx is a tool designed to convert the documentation for your package into a website.

For projects like open-source packages, it is hard to imagine prospective users or contributors persevering if the documentation is hard to follow, misleading or missing.

But for most projects, the user and maintenance-programmer described above will be somewhat closer to home: either yourself or members of your team. So, much like choosing informative names and writing well-structured code, spending some time to document your code will aid your future progress.

As with many things, documentation can be taken too far. Too much documentation and it may become a maintenance burden—becoming out-of-sync with the source code that it describes—this is a particularly common problem with code comments.

4.3 Everything with a script

Ideally, get the data in the rawest form possible. If you need to download external data, such as from the internet, it's best to do so via a script.² If that's not feasible, at least document the source of the data and when it was downloaded.

² Checkout the 'requests' package.

Write scripts for every stage of data processing — cleaning, renaming, filtering. You should *never* edit data files by hand. If you need to convert a data file from one form to another, use a script. Rather than opening the file in excel and saving each sheet by hand, do this with code.

All aspects of data cleaning should be in scripts. If you decide that you need to omit a couple of subjects, don't go into the data file and delete those rows, but rather write a script that will delete those subjects and create a separate, revised data file. If you find some errors in the data, don't go into the file and change it by hand, but rather make those changes via a script.³

³ Don't be afraid to rename column names!

4.4 Managing your dependencies

It is not uncommon to encounter a situation where you are working on two different projects, where each project requires a different version of the same package. To ensure that these two projects can coexist happily, you'll want to be able to isolate the environments in which they run.

It's also important to understand that many operating systems come bundled with a Python binary already. Making changes against this "system Python" is strongly discouraged, as it often leads to permission errors down-the-line (and, in fact, without administrative rights, you'll likely struggle to install additional packages against the system Python in the first place!). Additionally, any changes you make to the system Python may affect other users who rely on this Python. So, in addition to isolating your project's Python environments, you'll also want to isolate your projects from your operating system!

We can use the `virtualenv` tool from the command-line to create an isolated Python environment as:⁴

```
virtualenv -p python3 my-python-venv
```

Once you have created your virtual environment, you then need to *activate* it before you can use it. From a Windows machine you could activate the virtual environment created in Chunk 4.1 with:

```
.\my-python-venv\Scripts\activate
```

On a Linux / MacOS machine activating this virtual environment will look like:

```
source ./my-python-venv/bin/activate
```

Now that your virtual environment is activated you can `pip install` to your heart's content, knowing that these changes won't affect the Python environments used by your other projects.

Having installed the packages which your project requires, you can then capture these requirements in a text file with:

```
pip freeze > requirements.txt
```

The `requirements.txt` file should be regenerated each time you run a `pip install` command. A Python environment can later be restored from a `requirements.txt` file with:

⁴ A subset of the `virtualenv` tool has been integrated into Python since version 3.3, under the standard library's `venv` module. Although the `venv` module is convenient, it is not as featureful as `virtualenv`.

Chunk 4.1: Here the `-p` flag is used to ensure that `virtualenv` uses a Python 3 binary. The final argument specifies to store the machinery of this environment in a directory called 'my-python-venv'.

```
pip install -r requirements.txt
```

Sharing this `requirements.txt` file with others allows them to replicate the Python environment you used in your project.

Sometimes, however, to ensure that your project is reproducible, you may need to control more than just your project's *Python* environment. In some cases, you may also need to control elements of the operating system in which your Python environment will live. For example, your project may utilise Python packages which, to function, require additional libraries to be installed at the operating system level. You may even encounter Python packages which are only supported on particular operating systems!

When you need to exert such greater control over your project's environment, you'll want to make use of tooling such as Docker. Docker can be used to package up your Python code along with any necessary operating system dependencies. Your project will then be packaged as a Docker *image*, which can be distributed with others.

If you need to control your Python project's runtime environment down to the operating system level: then Docker may be the tool for you. A full introduction to Docker is outside the scope of this course, so you may wish to consult Docker's official documentation for working with Python if this is something you're interested in: <https://docs.docker.com/language/python/build-images/>.

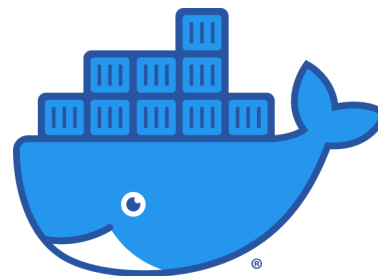


Figure 4.2: Docker's "Moby" logo may look cute, but don't be fooled: Docker is a *very* powerful technology.

4.5 Version control

Working with others in the same space can often get messy, especially when it comes to code. Imagine that you and your colleague Maria are working in the same file.

Do any of these file names look familiar to you?

```
code_v1.ipynb
code_v1b_add_plot.ipynb
code_v1b_add_plot_FINAL.ipynb
code_v1b_add_plot_FINAL_FINALv2_sept2020.ipynb
code_v1b_add_plot_FINAL_FINALv2_sept2020_Maria_edits.ipynb
```

If so, you need a version control system such as Git! What are the benefits?

- Backups of your files are stored remotely using an online host such as GitHub or GitLab.
- If you are working in a team, backups are stored on multiple locations.
- Git tracks the changes you make to files, so you have a record of what has been done, and you can revert to specific versions should you ever need to. This is called version control.
- Git allows changes by multiple people to be merged into a single source, thus improving collaborative working.

Git is a whole language in itself, and not something we can go into in this course. However it's definitely worth considering adding version control to your toolbox. There are a great number of resources out there on using git; we like the guide at: <https://realpython.com/python-git-github-intro/>

If you work in a large organisation, your I.T. department will almost certainly use some form of version control. Don't reinvent the maintenance wheel. Ask what they use!

When using git to manage your projects, ensure to keep large datasets out of your repository: having large datasets in your repository has the potential to significantly slow-down many git operations.⁵ It's also advisable to keep results of your analysis out of your repository: the code in your repository should include all the logic necessary to generate your results on demand.

⁵ If you *really* cannot avoid keeping large datasets out of your repository you should look into git LFS (large file storage).

In addition to keeping large datasets and results out of your repository, you should also keep an eye on cached and intermediary files and directories that may sometimes creep into your repository. These include items such as the `__pycache__` directory and `.pyc` files. Using a templated gitignore file, like the one at <https://github.com/github/gitignore/blob/main/Python.gitignore>, is a good way to keep such nuisance files out of your repo.

4.6 Write a Python package

As you write more and more functions, you may find yourself wanting to document and share your functions. By far the best way to do this in Python is by building a package. Writing a package can seem daunting at first. If you've browsed the source code of a popular Python package then you can be forgiven for feeling overwhelmed. But a package doesn't need to be the next 'sqlalchemy' or have the audience of 'pandas' to be worthwhile. If you have a handful of functions that *you* find useful, you can create a *personal Python package*.

As a developer you'll quickly realise that you have *a lot* of freedom to decide how best to structure your Python package. This freedom can be simultaneously liberating *and* overwhelming. To help structure your package we recommend starting from a template. Amongst other things, using a template:

- Avoids the paralysis-by-analysis induced by having so many structural decisions to make up-front;
- Helps newcomers familiar with the template you used, but unfamiliar with your project, get up-to-speed quicker;
- Doesn't waste your time reinventing the wheel;
- Allows you to concentrate on the important things (your code).

Cookiecutter is a handy command-line tool that can be used to instantiate projects from templates. To get your package off-the-ground we recommend using the cookiecutter tool along with the Python Data Science template described at <https://drivendata.github.io/cookiecutter-data-science/>:



```
cookiecutter \
  https://github.com/drivendata/cookiecutter-data-science
```

This template might initialise content that your project doesn't need. Feel free to remove these if you like: you can always add them back in later. A sample of the package structure created by this template looks like:

```
|-- data
|   |-- external      : Data from 3rd parties
|   |-- interim       : Intermediate data
```

```

| |-- processed      : Final data for models
| |-- raw           : Original data dump
|
|-- models          : Trained models
|
|-- reports         : Generated analysis
| |-- figures       : Generated graphics
|
|-- requirements.txt : For reproducibility
|
|-- src             : Source code
| |-- __init__.py   : Make a Python module
|
| |-- data          : Download or generate data
| | |-- make_dataset.py
| |
| |-- features      : Turn raw data into features
| | |-- build_features.py
| |
| |-- models        : Scripts to train models
| | |-- predict_model.py
| | |-- train_model.py
| |
| |-- visualization : Scripts to create visualizations
| |-- visualize.py

```

Within this package structure, you can add tests for your code. Tests help increase your confidence that your code works as expected and help during subsequent maintenance. If a function is generalised or rewritten, the tests should continue to pass. They also provide additional documentation regarding how your code should be used.

```

# Content of test/test_forecast.py
def weather_forecast(weather="rain"):
    return "Tomorrow will be: " + weather

def test_answer():
    assert "rain" in weather_forecast()

```

Such test files are *typically* stored in the `./tests/` directory of your package. However, the `pytest` package will find and run all files of the pattern `test*.py`, recursing from your current directory:

```

pytest
#> ===== test session starts
#> =====
#> platform linux -- Python 3.12.3, pytest-9.1.1,
#> pluggy-1.6.0
#> rootdir: /builds/jumpingrivers/training/python/best-
#> practices/notes

```

Chunk 4.2: The content in the file `test_forecast.py`.

```
#> plugins: anyio-4.14.2
#> collected 1 item
#>
#>
#> test_forecast.py .
#> [100%]
#>
#> ===== 1 passed in 0.02s
#> =====
```

If you want other people to be able to use your package, you'll want to release it under a software license. Software licenses are about two things: copyright, and protecting yourself from being held liable if your software screws something up somewhere down the line. There are lots of different software licenses to choose from.⁶ At Jumping Rivers we tend to use GNU General Public License (GPL3), but it's worth reading about the different ones and choosing the appropriate one for your project.

⁶ See <https://tldrlegal.com/> for a user-friendly overview.

Working Collaboratively

One of the best ways you can improve your coding style, is by learning from other programmers. In this chapter we'll look at techniques you can use for working well together and learning from each other.

5.1 Code review

Code review is simply the process of someone else reviewing code that you have written. The reviewer might make comments, offer constructive criticism and resolve small issues. Code reviews are beneficial because they can help eliminate bugs, improve code readability and share knowledge across a team.

If you are using a version control system¹, the code should be reviewed pre-merge, i.e. before it is merged into the main branch. If you are using a git platform like GitLab or GitHub, it is easy to build code review into the merge request process. It is also possible to assign a *code owner*² who is responsible for signing off on any changes to the code base.

If you are asked to review someone's code you should try to respond promptly, explain your reasoning and balance being directive with flexibility.³ If you aren't in the middle of deep work, you should respond to a code review request shortly after it comes in. Otherwise wait until a natural break in your day. As a rule of thumb, we expect code review requests to be responded to within a day. If you are too busy to do a full review, you may be able to provide some initial broad comments or suggest an alternative reviewer.⁴

If you have comments to make, it's important to explain why those changes are necessary.

- **Bad:** "This is slow"
- **Good:** "As the data is so large, running can slow things down. Consider using caching to speed things up"

If the code review is too large, ask the developer to split it up. If the code is too complex, ask them to make it simpler to understand rather than asking them to explain the complexity to you.⁵

Sometimes it's easier to fix something than explain what is wrong. For example, a typo in a variable name, or a violation of your internal code style. In these cases, you might choose to just fix the code yourself rather than asking the developer to fix it. In some cases giving explicit fixes with just pointing out problems and letting the developer decide

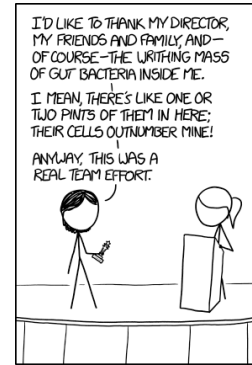


Figure 5.1: <https://xkcd.com/1543/>

¹ And you should!

² GitLab allows different owners for code. In the CODEOWNERS file, we can list usernames and the associated directories.

³ At Jumping Rivers, we have *merge-monday* for website updates. This allows multiple non-urgent merge requests to be reviewed at once.

⁴ The key thing is communication and expectations. If you can't look at something in the near future—let people know!

⁵ The common joke is the larger the merge request, the fewer the number of comments.

Bad	Good
Maria, why did you use a for loop here? There is obviously a much better way to answer that	Have you considered using numpy—it could simplify the code.
Why did you use threads here when there's obviously no benefit to be gained from concurrency?	The concurrency model here is adding complexity to the system without any actual performance benefit that I can see. Because there's no performance benefit, it's best for this code to be single-threaded instead of using multiple threads.

Table 5.1: Good and bad code reviews

For example a reviewer might respond as follows.

- I've fixed the linting issues and clarified some variables names⁶.
- Consider using `the_string.lower()` to pre-process the naming field
- The script is getting long, can the code be modularised?

⁶ It's essential to have a group policy for this to avoid `snake_case` vs `camelCase` wars.

Finally be kind. Sharing code can sometimes make you feel vulnerable and it's important to be courteous and kind. One trick to help writing constructive code reviews to is to only make comments about the code and never make comments about the individual developer.

If you see something nice in the code being reviewed do tell the programmer. Too often, code review just focus on mistakes, but they should offer encouragement and appreciation for good practices as well.

Using your criticism points

If a reviewer makes multiple suggestions, this places a significant burden on the reviewer's time and can also be demoralising the for the original author. We can think of this as *reviewer points*; a reviewer can only spend these points before other issues creep in. A good way of saving these points for key issues is to use continuous integration (CI). This process is useful for picking up on minor issues, like linting, thereby saving the *reviewer points* for other more important topics.

5.2 *Pair programming*

A real-time form of code review is pair programming, where two programmers sit together while writing code. One programmer, *the driver* actually writes the code and focusses solely on the task at hand. The other, called *the navigator* does not type, and is free to detect bugs, issues, design and think about the wider structure of the project.

Pair programming is an efficient way of detecting bugs, writing better code and understanding the task at hand better. Having two people working simultaneously tends to mean you don't get "lost" in the code, and having another person to work with tends to keep both developers on track and productive.

Pairing also helps with knowledge transfer across a team. You always have someone else who understands the code base. It's a great way to train up junior members of the team, and a good way to get to know your colleagues better.

I check Twitter less when I pair program with you. Anonymous Data Scientist at Jumping Rivers.

Pairing is also nice when you are working remotely!

There can be some downsides of pair programming. Sometimes pair programming can feel a little slow and awkward at the start, especially if you do not know the other programmer well. It can also be tiring and feel intrusive for programmers who find social interactions tiring. Pair programming is a great tool for the right tasks. We recommend that teams use pair programming when bringing a junior team member up to speed and when tackling particularly complex problems.

5.3 Asking great questions

We all get stuck sometimes and need to ask for help. That's why Q&A forums such as Stack Overflow are so popular.⁷ Programmers regularly visit sites like these weekly to ask for help or look at the previous answers.

If you can't find an existing question that helps you out, you might want to ask others for help. This could be explaining the problem to a colleague or putting out a question in public like on Stack Overflow. But how do you explain your problem in the simplest way so that others will help you? The answer is to create a *reprex*, or *reproducible example*. The goal of a reprex is to package your problematic code in such a way that other people can run it and feel your pain. Then, hopefully, they can provide a solution and put you out of your misery. There are two parts to creating a reprex, making it reproducible and making it simple.

It's important that someone trying to help you can replicate your problem exactly, this means understanding what packages you have loaded and any existing variables you might have in your workspace. You then have to make it simple. Remove any code which isn't directly related to the issue you are having. Instead of using your actual data, create a smaller example dataset. Often in the process of writing the reprex, you actually solve your own problem.

Reproducible examples

Suppose you have an issue with some plotting code:

```
import datetime
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import my_constants

my_data = pd.read_csv("my_data.csv")
my_data = my_data[my_data.age > my_constants.min_age]

plot1 = plt.plot(
    my_data["age"], my_data["metallicity_1"]
)
plot2 = plt.plot(
    my_data["age"], my_data["metallicity_2"]
)
```

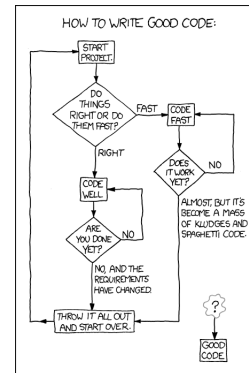


Figure 5.2: <https://xkcd.com/844/>

⁷ A good way of learning how to ask questions, is to try and answer other existing questions. You quickly realise what makes a good question.

Chunk 5.1: The issue you experience in this code is difficult for another user to replicate.

```
plt.legend([plot1, plot2], ["sample 1", "sample 2"])
plt.show()
```

Your plot appears, but the legend you tried to add is missing!

You're not sure how to fix this problem, and so you decide to ask for help from someone else. *However*, your code doesn't make for a good reprex (reproducible example) as:

1. Your code doesn't run *as-is*; for someone else to run your code you'd need to share `my_constants.py` and `my_data.csv` with them too.
2. Modules unrelated to the issue are imported;
3. Operations unrelated to the issue are performed, e.g., filtering the data by `age` ;
4. The code contains technical and potentially distracting names, e.g., `metallicity` .

The key idea is to allow another user to reproduce your issue, using the *minimal* amount of code possible.⁸

Your code in Chunk 5.1 can be made a better reprex by removing unnecessary imports and providing a small example `DataFrame` :

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame(
    {
        "x": [1, 2, 3],
        "y_1": [2, 3, 5],
        "y_2": [2, 2, 4],
    }
)

plot1 = plt.plot(df["x"], df["y_1"])
plot2 = plt.plot(df["x"], df["y_2"])
plt.legend([plot1, plot2], ["sample 1", "sample 2"])
plt.show()
```

The example is now completely reproducible and the problem is clear; this helps others help you!

The reprexy package

Writing a good reprex can be hard work. Luckily, there is a Python package to lend a helping hand. When you want to make a reprex, you can use the `reprexy` package:⁹

```
import reprexy
```

To use this package, write a bit of code and copy it to your clipboard with `Ctrl + C` or a right-click. Type `reprexy.reprex()` in the Python Console. This will render a reprex, including any outputs from executing the code (even figures!). The reprex will be ready and waiting on your clipboard, so you can paste it into, say, a GitHub issue (for Stack Overflow, you should use `reprexy.reprex(venue="so")`).

⁸ A synonym for reprex is MWE: minimal working example.

⁹ <https://pypi.org/project/reprexy/>

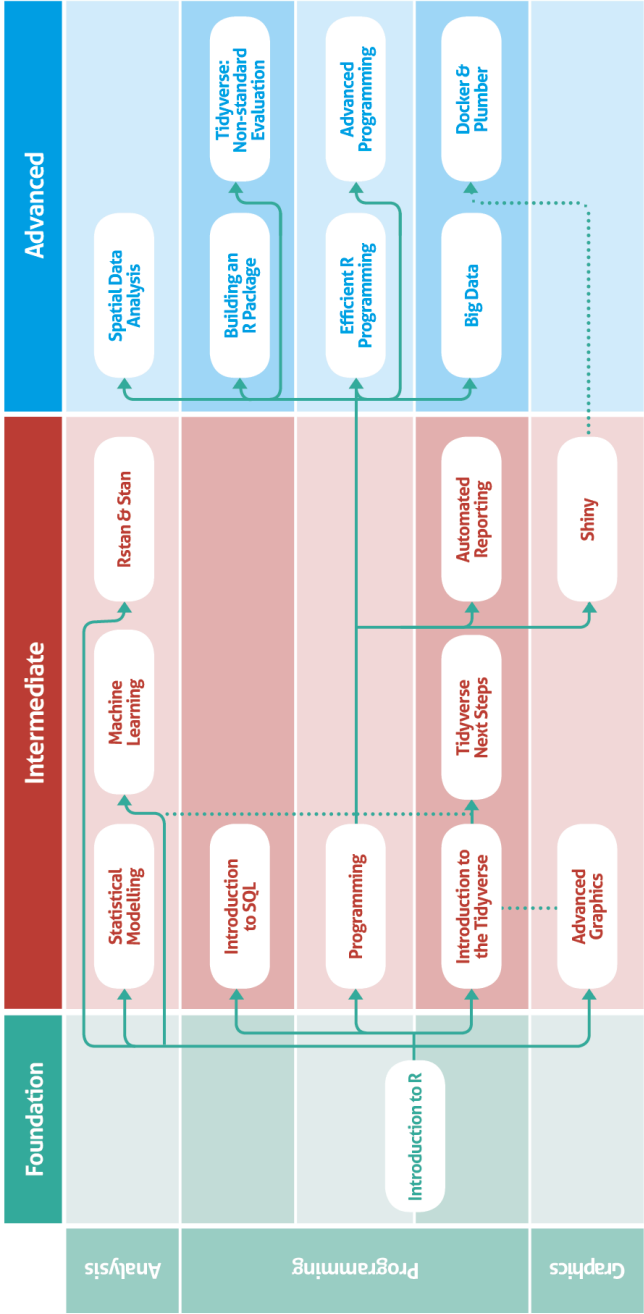
5.4 *How to work in a group*

Communication and agreed ground rules are essential for successful group working. For example, should every project have a README? If so, then enforcing a standard rule reduces overall friction, than leaving it to individual opinions.

- Create an overview of your project. (README)
- Create a shared “to-do” list (issue tracker)
- Decide on communication strategies
- Make small changes with frequent feedback

Once a team grows beyond a certain size, it becomes difficult to keep track of what needs to be reviewed, or of who’s doing what. Teams can avoid a lot of duplicated effort if they use an issue tracking tool to maintain a list of tasks to be performed and bugs to be fixed. This helps avoid duplicated work and makes it easier for tasks to be transferred to different people. Free repository hosting services like GitHub include issue tracking tools.

Training Course Dependencies





OUR PARTNERS